

FlowKit 관련 선행연구 조사 보고서

부제: Block-based Context Editing & Branching Chatbot 구현을 위한 Context Engineering 및 HCI 선행연구 검토

작성 기준: 2026 년 5 월 7 일 오프라인 회의 정리 및 FlowKit 서비스 방향

작성일: 2026 년 5 월 20 일

작성 주체: 인공지능응용학과 캡스톤디자인 팀

1. 조사 목적

본 문서는 캡스톤디자인 프로젝트 **FlowKit**의 기획 및 구현 방향을 정당화하기 위해 관련 선행연구를 정리한 보고서이다. 2026 년 5 월 7 일 회의 이후 기존의 광범위한 문서 워크스페이스형 기획은 축소하고, 최종 구현 방향을 **대화 맥락을 사용자가 블록 단위로 선택·편집·분기할 수 있는 Context 관리형 챗봇 서비스**로 정리하였다.

FlowKit의 핵심 문제의식은 다음과 같다.

- 기존 챗봇은 이전 대화 전체를 시간순으로 누적해 다음 응답의 맥락으로 사용하는 경향이 있다.
- 사용자가 AI의 긴 답변을 모두 읽거나 동의하지 않았더라도, 해당 답변 전체가 이후 대화에 암묵적으로 반영될 수 있다.
- 사용자가 특정 맥락을 삭제하거나 수정하고 싶어도, 일반적인 채팅 UI에서는 모델이 실제로 무엇을 기억하고 있는지 확인하거나 조작하기 어렵다.
- 단순한 새 채팅 또는 일반적인 브랜치 기능은 특정 시점까지의 대화 전체를 가져오는 방식에 가깝기 때문에, 사용자가 원하는 일부 맥락만 선택적으로 이어가는 데 한계가 있다.

따라서 FlowKit은 LLM 자체를 새로 학습시키는 프로젝트라기보다, **LLM에게 전달되는 Context를 사용자 인터페이스 차원에서 선택·정제·분기하는 Human-in-the-loop Context Management 시스템**으로 정의하는 것이 적절하다.

2. FlowKit의 AS-IS / TO-BE 정리

2.1 AS-IS: 기존 챗봇의 대화 맥락 처리 방식

기존 챗봇은 일반적으로 사용자의 입력과 AI 응답을 하나의 선형 대화 기록으로 누적한다. 사용자가 후속 질문을 하면 시스템은 이전 대화의 상당 부분을 그대로 참조하거나 요약된 상태로 반영한다. 이 구조는 짧은 질의응답에는 편리하지만, 대화가 길어질수록 다음 문제가 발생한다.

- 읽지 않은 AI 응답도 이후 맥락에 포함된다.
- 폐기된 아이디어나 잘못된 설명이 후속 응답에 영향을 줄 수 있다.
- 사용자는 어떤 맥락이 현재 응답에 반영되는지 확인하기 어렵다.
- 사용자가 특정 정보만 남기거나 특정 정보만 제외하는 세밀한 제어가 어렵다.
- 분기 기능이 있더라도 “특정 시점까지의 전체 대화” 중심이지, “선택한 의미 단위의 Context” 중심은 아니다.

2.2 TO-BE: FlowKit 의 목표 상태

FlowKit 은 대화를 **Message Block** 단위로 나누고, 사용자가 필요한 블록만 선택해 우측 Context 편집 패널에서 정제한 뒤 다음 대화 또는 새로운 브랜치에 적용할 수 있게 한다.

핵심 기능은 다음과 같다.

- 1. 사용자 질문과 AI 응답을 메시지 블록 단위로 표시한다.
- 2. 사용자는 다음 질문에 반영할 블록만 선택한다.
- 3. 선택된 블록은 우측 Context 편집 패널에 모인다.
- 4. 사용자는 “핵심만 요약”, “불필요한 내용 제거”, “초보자용으로 재작성” 등 자연어 지시로 Context 를 정제한 다.
- 5. 정제된 Context 미리보기를 확인한 뒤, 해당 Context 만 기반으로 다음 질문을 이어간다.
- 6. 필요할 경우 선택·정제된 Context 를 기반으로 새로운 브랜치를 생성한다.
- 7. 이전 메시지 수정 및 응답 버전 관리를 통해 대화 흐름을 부분적으로 교정할 수 있다.

이 구조에서 FlowKit 의 핵심 가치는 “더 좋은 답변을 한 번에 생성하는 것”보다 **AI 가 무엇을 기억하고 무엇을 잊어야 하는지 사용자가 직접 관리하는 것**에 있다.

3. 논문 선정 기준

본 조사는 교수님 피드백에서 요구된 “Key Paper” 탐색을 목적으로 하며, 다음 기준으로 논문을 선정하였다.

- 1. **프로젝트 직접 관련성**: Context 를 명시적 객체로 보고 사용자가 조작하는 관점이 있는가?
- 2. **브랜치 및 대화 흐름 관리**: 선형 대화가 아니라 선택·분기·재사용 가능한 대화 구조를 다루는가?
- 3. **문제 정의 근거**: 긴 Context 또는 다중 턴 대화에서 LLM 성능이 저하된다는 근거를 제공하는가?
- 4. **구현 참고 가능성**: FlowKit 의 UI/UX, 시스템 구조, 평가 지표로 변환 가능한가?
- 5. **공신력**: 학술지/학회 게재 여부, arXiv preprint 여부, 저자 및 기관, 코드/데이터 공개 여부를 고려하였다.

4. 핵심 선행연구 요약표

우선순위	논문	프로젝트 내 역할	선정 이유	공신력 및 주의점
1	Mixed-Initiative Context: Structuring and Managing Context for Human-AI Collaboration (Li et al., 2026)	Mother paper	Context 를 명시적·구조적·조작 가능한 상호작용 객체로 정의한다. FlowKit 의 “Context 편집 패널”과 가장 직접적으로 연결된다.	arXiv preprint. HCI 관점의 직접 관련성은 높으나, 대규모 정량 검증은 제한적이다.
2	Context Branching for LLM Conversations (Nanjundappa &	Father paper	checkpoint, branch, switch, inject 같은 대화 버전관리 primitive 를 제시한다. FlowKit	arXiv preprint. 소프트웨어 개발 시나리오 중심이므로 일반 챗봇 사용으로 확장할 때 조

우선순위	논문	프로젝트 내 역할	선택 이유	공신력 및 주의점
	Maaheshwari, 2025)		의 선택형 브랜치 기능과 직접 연결된다.	정이 필요하다.
3	Lost in the Middle: How Language Models Use Long Contexts (Liu et al., 2024)	문제 정의 근거	긴 Context 를 입력해도 관련 정보의 위치에 따라 성능이 저하될 수 있음을 보인다.	TACL 2024 게재 논문. 공신력이 가장 높고 문제정의 근거로 적합하다.
4	LLMs Get Lost In Multi-Turn Conversation (Laban et al., 2025)	다중 턴 문제 근거	다중 턴 대화에서 모델이 초기 가정과 이전 응답에 과도하게 의존하며 성능이 저하될 수 있음을 보인다.	arXiv preprint. 대규모 시뮬레이션 기반이므로 평가 설계 참고 가치가 크다.
5	MemGPT: Towards LLMs as Operating Systems (Packer et al., 2023/2024)	시스템 구조 참고	제한된 context window 를 memory tier 로 관리하는 virtual context management 관점을 제시한다.	arXiv preprint. FlowKit 은 자동 메모리보다 사용자 조작 UI에 초점을 둔다는 차이를 명확히 해야 한다.
6	A Survey of Context Engineering for Large Language Models (Mei et al., 2025)	용어·분류 체계 참고	Context Engineering 을 prompt engineering 보다 넓은 개념으로 정리하고, retrieval, processing, management, memory system 을 분류한다.	arXiv survey. 분야 지도 작성에 유용하나, 개별 구현 논문은 별도 검토가 필요하다.
보조	ConstraintBench: Benchmarking LLM Constraint Reasoning on Direct Optimization (Tso et al., 2026)	초기 문제의식 참고	LLM 이 제약을 안정적으로 만족하지 못한다는 초기 아이디어의 근거가 된다.	현재 FlowKit 의 핵심 주제와는 직접 관련성이 낮아 Key Paper 보다는 보조 참고문헌으로 두는 것이 적절하다.

5. 주요 논문별 상세 검토

5.1 Li et al. (2026), Mixed-Initiative Context

논문 개요

이 논문은 Human-AI Collaboration 에서 Context 를 더 이상 숨겨진 대화 기록이나 모델 입력의 부수물로 보지 않고, **명시적이고 구조화되어 있으며 조작 가능한 상호작용 객체**로 재정의한다. 사용자는 Context 를 직접 포함·제외·수정·재조직할 수 있고, AI 는 브랜치 제안, 패턴 추출, 구조 추천 등을 제안하되 최종 반영 권한은 사용자에게 둔다.

선택 이유

FlowKit 의 핵심 아이디어는 “사용자가 대화 Context 를 직접 선택하고 정제한다”는 것이다. 이는 해당 논문의 Mixed-Initiative Context 개념과 거의 동일한 문제의식을 공유한다. 특히 다음 부분이 직접적으로 대응된다.

Mixed-Initiative Context	FlowKit 대응 기능
Context 를 조작 가능한 객체로 취급	메시지 블록 선택 및 편집
Context unit 의 포함·제외·수정	체크박스 기반 블록 선택, 빠른 편집, 자연어 편집 지시
Context 구조의 branching, returning, pattern extraction	선택 Context 기반 브랜치 생성
AI 가 제안하되 사용자가 승인	정제 Context 미리보기 후 적용

프로젝트 적용 방식

이 논문은 FlowKit 의 이론적 기반으로 가장 적합하다. 제안서나 발표에서는 다음처럼 표현할 수 있다.

본 프로젝트는 Li et al.의 Mixed-Initiative Context 개념을 캡스톤 수준의 서비스 프로토타입으로 구현하는 시도이다. 기존 채팅 UI 가 Context 를 암묵적·선형적으로 누적했다면, FlowKit 은 Context 를 블록 단위로 명시화하고 사용자가 직접 선택·정제·분기할 수 있게 한다.

한계 및 주의점

해당 논문은 HCI 관점의 개념과 프로브 시스템에 초점이 있으며, 대규모 성능 벤치마크나 일반화된 정량 평가보다는 사용자의 조작 경험과 설계 공간을 탐색하는 성격이 강하다. 따라서 FlowKit 도 모델 성능 향상만을 주장하기보다 **사용자 제어감, 맥락 명료성, 불필요한 Context 감소, 후속 응답의 초점성** 등을 평가하는 것이 더 자연스럽다.

5.2 Nanjundappa & Maaheshwari (2025), ContextBranch

논문 개요

이 논문은 LLM 대화를 버전 관리 관점으로 다루며, ContextBranch 라는 대화 관리 시스템을 제안한다. 주요 primitive 는 다음 네 가지이다.

1. checkpoint: 현재 대화 상태 저장

- 2. branch: 특정 상태에서 새로운 탐색 경로 생성
- 3. switch: 다른 대화 경로로 전환
- 4. inject: 한 경로의 유용한 정보를 다른 경로에 선택적으로 삽입

논문은 선형 대화가 누적될수록 context pollution 이 발생하며, 사용자가 새 채팅을 시작하면 오히려 유용한 맥락까지 잃는다는 문제를 제기한다.

선택 이유

FlowKit 의 핵심 기능 중 하나는 **선택형 브랜치 생성**이다. 일반적인 브랜치가 특정 시점까지의 대화 전체를 복사하는 방식이라면, FlowKit 은 사용자가 선택한 메시지 블록 또는 정제된 Context 만으로 새로운 대화 흐름을 만든다. ContextBranch 는 이 기능을 연구적으로 설명할 수 있는 가장 직접적인 참고문헌이다.

프로젝트 적용 방식

ContextBranch 의 primitive 를 FlowKit 에 맞춰 단순화하면 다음과 같다.

ContextBranch primitive	FlowKit 구현 해석
checkpoint	현재 대화에서 선택한 블록 집합 저장
branch	선택 Context 기반 새 대화 흐름 생성
switch	좌측 브랜치 목록에서 흐름 전환
inject	정제된 Context 를 다음 질문 입력에 적용

FlowKit 은 ContextBranch 보다 구현 범위를 줄여, “프로그래밍 탐색”이 아니라 “일반 학습·기획·정리 대화”에서도 사용할 수 있는 블록 기반 대화 브랜치 UI 로 해석할 수 있다.

한계 및 주의점

ContextBranch 는 주로 exploratory programming 상황에 초점을 둔다. 따라서 FlowKit 이 일반 챗봇 사용자를 대상으로 한다면, 실험 태스크를 프로그래밍에 한정하지 않고 학습, 정보 정리, 문서 기획 등의 일상적인 장기 대화 태스크로 재구성하는 것이 좋다.

5.3 Liu et al. (2024), Lost in the Middle

논문 개요

이 논문은 긴 Context 를 처리할 수 있는 LLM 이라도, 입력 Context 안에서 관련 정보가 어디에 위치하느냐에 따라 성능이 크게 달라질 수 있음을 보였다. 특히 관련 정보가 Context 의 앞부분이나 끝부분에 있을 때보다 중간에 있을 때 모델 성능이 떨어지는 경향이 보고되었다.

선택 이유

FlowKit의 문제 정의는 “긴 대화 전체를 그대로 넣는 것이 항상 좋은 것은 아니다”라는 주장에 기반한다. Lost in the Middle은 이 주장을 뒷받침하는 강한 근거가 된다.

프로젝트 적용 방식

이 논문은 다음 주장에 활용할 수 있다.

긴 context window가 있다고 해서 모든 대화 기록을 그대로 넣는 것이 최선은 아니다. 모델은 긴 Context 안의 관련 정보를 항상 안정적으로 활용하지 못하므로, 사용자가 필요한 Context를 명시적으로 선택하고 정리하는 인터페이스가 필요하다.

공신력

TACL 2024에 게재된 peer-reviewed 논문이며, 본 리서치에서 문제 정의 근거로 가장 안정적으로 활용할 수 있다.

5.4 Laban et al. (2025), LLMs Get Lost In Multi-Turn Conversation

논문 개요

이 논문은 단일 턴에서 완전한 요구사항을 제공받는 경우와, 여러 턴에 걸쳐 요구사항이 점진적으로 드러나는 경우의 LLM 성능을 비교한다. 논문은 다중 턴 대화에서 모델 성능이 평균적으로 크게 저하될 수 있고, 모델이 초기에 잘못된 가정을 하거나 이전 응답에 과도하게 의존하는 경향을 보인다고 설명한다.

선택 이유

FlowKit은 단순히 긴 Context 문제만 다루는 것이 아니라, **여러 턴에 걸쳐 대화가 진행되면서 원치 않는 맥락이 누적 되는 문제**를 다룬다. 따라서 이 논문은 FlowKit의 “다중 턴 대화 맥락 관리” 필요성을 설명하는 데 적합하다.

프로젝트 적용 방식

이 논문의 평가 방식은 FlowKit의 실험 설계에 직접 참고할 수 있다. 예를 들어 하나의 완전한 요구사항을 여러 메시지로 나누어 제시하고, baseline 챗봇과 FlowKit 방식에서 최종 응답의 요구사항 반영률, 불필요한 가정 반영 여부, 응답 초점성을 비교할 수 있다.

한계 및 주의점

해당 논문은 시뮬레이션 기반 평가에 초점이 있으며, FlowKit처럼 사용자가 UI를 통해 직접 Context를 조작하는 상황까지 검증한 것은 아니다. 따라서 본 프로젝트에서는 논문의 문제의식을 가져오되, HCI형 사용자 평가를 병행하는 방식이 적절하다.

5.5 Packer et al. (2023/2024), MemGPT

논문 개요

MemGPT는 제한된 context window 문제를 해결하기 위해 LLM을 운영체제처럼 보고, memory tier를 활용하는 virtual context management 구조를 제안한다. 핵심은 LLM이 필요한 정보를 장기·단기 메모리 사이에서 이동시키며 긴 대화나 문서 분석을 처리하도록 하는 것이다.

선택 이유

FlowKit도 “모든 대화를 그대로 모델에 넣는 것이 아니라 필요한 Context만 구성한다”는 점에서 MemGPT와 문제 의식을 공유한다. 다만 MemGPT가 agent-managed memory architecture에 가깝다면, FlowKit은 **user-managed context interface**에 가깝다.

프로젝트 적용 방식

MemGPT는 시스템 아키텍처 설명에 활용할 수 있다.

- raw conversation: 원본 대화 로그
- selected blocks: 사용자가 선택한 활성 Context
- refined context: AI 편집 지시를 거쳐 정제된 Context
- active prompt: 다음 응답 생성을 위해 모델에 전달되는 최종 Context

이처럼 FlowKit의 내부 구조를 “원본 로그와 활성 Context를 분리하는 구조”로 설명할 수 있다.

한계 및 주의점

MemGPT는 사용자가 직접 Context 블록을 선택·편집하는 UI보다는, 시스템이 자동으로 메모리를 관리하는 구조에 초점을 둔다. 따라서 FlowKit의 차별점은 “자동 메모리 관리”가 아니라 “사용자에게 보이는 Context 편집 인터페이스”라고 명확히 해야 한다.

5.6 Mei et al. (2025), A Survey of Context Engineering for Large Language Models

논문 개요

이 survey는 Context Engineering을 단순한 Prompt Engineering보다 넓은 개념으로 정의한다. Context Engineering은 모델 입력에 포함되는 정보의 검색, 생성, 처리, 관리, 메모리 시스템, RAG, tool use, multi-agent 구조 등을 포함하는 포괄적 분야로 정리된다.

선택 이유

FlowKit의 프로젝트명을 단순한 “챗봇 UI”가 아니라 “Context Engineering 기반의 사용자 주도 Context 관리 인터페이스”로 설명하려면, 해당 survey가 용어와 분야 정리를 제공한다.

프로젝트 적용 방식

제안서에서는 다음처럼 사용할 수 있다.

FlowKit은 Context Engineering의 여러 하위 영역 중 특히 context management와 context processing을 사용자 인터페이스 차원에서 다루는 프로젝트이다. 기존 연구가 RAG나 memory system 등 시스템 내부의 context 자동 관리에 초점을 두었다면, FlowKit은 사용자가 직접 active context를 선택·정제·분기한다는 점에서 HCI적 차별성이 있다.

한계 및 주의점

Survey 논문은 분야 전체를 넓게 정리하는 데 유용하지만, FlowKit의 직접 구현 방식은 Mixed-Initiative Context와 ContextBranch 쪽에서 더 구체적으로 가져오는 것이 좋다.

5.7 Tso et al. (2026), ConstraintBench

논문 개요

ConstraintBench는 LLM이 제약 조건이 있는 최적화 문제에서 직접 해를 생성할 수 있는지 평가하는 benchmark이다. 논문은 최신 모델도 제약 만족에 어려움을 겪으며, feasibility가 주요 병목임을 보고한다.

FlowKit과의 관련성

이 논문은 프로젝트 초기 단계의 “LLM은 복잡한 제약을 안정적으로 지키기 어렵다”는 문제의식에는 도움이 된다. 그러나 5월 7일 회의 이후 FlowKit은 제약 최적화 또는 프롬프트 중개 시스템이 아니라, 사용자가 대화 Context를 선택·편집·분기하는 서비스로 방향이 정리되었다. 따라서 ConstraintBench는 핵심 Key Paper라기보다 다음 정도의 보조 참고문헌으로 두는 것이 적절하다.

- LLM이 복잡한 조건을 항상 안정적으로 처리하지 못한다는 배경 근거
- 자동 생성 결과를 맹신하지 않고 사용자가 직접 확인·편집해야 한다는 Human-in-the-loop 설계의 보조 근거
- 향후 평가에서 “제약 준수율”을 측정할 경우 참고 가능한 benchmark 예시

6. 종합 분석: FlowKit의 연구적 포지셔닝

선행연구를 종합하면 FlowKit은 다음과 같이 포지셔닝할 수 있다.

FlowKit은 LLM 자체의 추론 성능을 개선하는 모델 연구가 아니라, 다중 턴 대화에서 누적되는 Context를 사용자가 블록 단위로 선택·정제·분기할 수 있게 하는 HCI 기반 Context Engineering 서비스이다.

기존 연구 대비 FlowKit의 차별점은 다음과 같다.

1. **Context의 명시화**
기존 챗봇에서는 Context가 숨겨진 상태로 누적된다. FlowKit은 메시지 블록과 Context 패널을 통해 현재 활용할 맥락을 사용자가 직접 볼 수 있게 한다.
2. **Context의 선택 가능성**
전체 대화가 아니라 필요한 메시지 블록만 다음 질문에 반영한다.

- 3. **Context의 재작성 가능성**
선택한 블록을 그대로 쓰지 않고, AI 편집 지시를 통해 요약·삭제·변환한 뒤 적용한다.
- 4. **Context 기반 브랜치**
특정 시점까지의 대화 전체가 아니라, 사용자가 선택한 의미 단위의 Context를 기반으로 새 대화 흐름을 생성한다.
- 5. **사용자 통제권 보장**
AI가 자동으로 모든 맥락을 결정하지 않고, 사용자가 미리보기 후 적용 여부를 결정한다.

따라서 본 프로젝트는 다음 세 축을 결합한 캡스톤 프로젝트로 설명할 수 있다.

축	설명	대표 참고 논문
HCI / 사용자 제어	Context를 명시적이고 조작 가능한 객체로 전환	Mixed-Initiative Context
대화 구조 / 브랜치	대화를 선형 로그가 아닌 분기 가능한 구조로 관리	ContextBranch
LLM Context 문제	긴 Context와 다중 턴에서 성능·신뢰성 저하 발생	Lost in the Middle, LLMs Get Lost In Multi-Turn Conversation

7. 평가 방향 제안

FlowKit은 모델 자체의 정확도 향상만으로 평가하기 어렵다. 따라서 교수님께 제안할 평가 방향은 **정량 지표 + 사용자 평가**의 혼합 방식이 적절하다.

7.1 정량 평가 지표

평가 항목	측정 지표	측정 방법
Context 절감	선택 전후 메시지 수, token 수, context compression ratio	동일 대화에서 전체 history 사용 대비 선택·정제 Context 사용 시 입력 token 감소량 측정
응답 초점성	불필요한 주제 혼입 수, 관련 문장 비율	사전에 정한 target context와 무관한 내용이 응답에 포함되는지 평가
지시 반영률	사용자가 선택한 Context 요소 중 응답에 반영된 항목 비율	체크리스트 기반 수동 평가 또는 LLM-as-judge 보조 평가
Latency	TTFT, 전체 응답 시간	전체 history 입력 대비 정제 Context 입력의 응답 시간 비교
브랜치 품질	브랜치 목적과 응답 일치도	선택 Context 기반 브랜치와 일반 브랜치의 후속 응답 비교

7.2 사용자 평가 지표

평가 항목	예시 질문
제어감	“내가 AI 가 기억할 내용을 통제하고 있다고 느꼈는가?”
맥락 이해 용이성	“현재 대화가 어떤 맥락을 기반으로 이어지는지 파악하기 쉬웠는가?”
사용 편의성	“블록 선택과 Context 편집 과정이 부담스럽지 않았는가?”
결과 신뢰도	“정제된 Context 를 확인한 뒤 후속 응답을 더 신뢰할 수 있었는가?”
재사용 의향	“긴 대화나 학습·문서 작업에서 이 기능을 다시 사용할 의향이 있는가?”

7.3 실험 설계 예시

비교군은 다음과 같이 설정할 수 있다.

- Baseline A: 일반 선형 챗봇 방식. 이전 대화 전체를 그대로 유지한다.
- Baseline B: 단순 새 채팅 또는 일반 브랜치 방식. 특정 시점 기준으로 대화를 분기한다.
- Proposed: FlowKit 방식. 사용자가 필요한 블록만 선택하고 정제한 뒤 후속 질문 또는 브랜치를 생성한다.

태스크는 학부 캡스톤 범위를 고려해 다음처럼 단순화할 수 있다.

1. 학습 설명 태스크: 긴 설명 중 필요한 개념만 선택해 후속 질문
2. 기획 아이디어 태스크: 여러 후보 아이디어 중 일부만 선택해 새 방향으로 브랜치
3. 문서 요약·정리 태스크: 긴 AI 응답에서 핵심만 정제해 다음 문서 초안 생성

평가 규모는 초기에는 5~10 명 수준의 소규모 사용자 테스트로 시작하고, 2 학기에는 베타 테스터를 늘려 사용자 만족도와 정량 로그를 함께 수집하는 방식이 현실적이다.

8. 교수님께 제안할 Key Paper 선정안

본 프로젝트의 핵심 Key Paper 는 다음 두 편으로 제안한다.

8.1 Mother Paper: Mixed-Initiative Context

이 논문은 FlowKit 의 가장 중요한 이론적 기반이다. Context 를 숨겨진 모델 입력이 아니라 사용자가 직접 조작 가능한 상호작용 객체로 정의하기 때문이다. FlowKit 의 메시지 블록 선택, 우측 Context 편집 패널, 정제 Context 미리보기, 사용자 승인 후 적용 기능은 모두 이 논문의 관점에서 설명할 수 있다.

8.2 Father Paper: ContextBranch

이 논문은 FlowKit의 브랜치 기능을 설명하는 데 가장 직접적인 근거가 된다. FlowKit은 ContextBranch의 version control 관점을 일반 챗봇 UI에 맞게 단순화한 형태로 구현할 수 있다. 특히 “선택한 Context 만 기반으로 브랜치를 만든다”는 점이 핵심 차별점이다.

8.3 Supporting Papers

- Lost in the Middle: 긴 Context 전체를 무작정 넣는 방식의 한계를 설명한다.
- LLMs Get Lost In Multi-Turn Conversation: 다중 턴 대화에서 모델이 이전 가정과 불필요한 맥락에 갇힐 수 있음을 설명한다.
- MemGPT: context window와 memory management의 시스템적 배경을 제공한다.
- A Survey of Context Engineering: 프로젝트를 Context Engineering 범주로 설명할 수 있는 용어 체계를 제공한다.

9. 지도교수 면담 시 질문 제안

1. 본 프로젝트를 “모델 성능 개선 연구”가 아니라 “HCI 기반 Context Engineering 서비스 구현”으로 포지셔닝해도 적절한지 여쭙고 싶습니다.
2. Key Paper를 Mixed-Initiative Context와 ContextBranch 두 편으로 잡고, Lost in the Middle과 Multi-Turn Conversation 논문을 문제 정의 근거로 사용하는 구성이 적절한지 여쭙고 싶습니다.
3. 평가 방식을 대규모 benchmark보다 소규모 사용자 테스트, context 절감률, latency, 사용자 제어감, 응답 초점성 중심으로 잡아도 캡스톤 결과물로 충분한지 여쭙고 싶습니다.
4. 1학기 산출물은 UI 시안, 요구사항정의서, 기능 명세서, 세부 설계서 중심으로 두고, 2학기에 구현 및 사용자 테스트를 진행하는 일정이 적절한지 여쭙고 싶습니다.
5. FlowKit의 MVP 범위를 “기본 채팅 화면, 메시지 블록 선택, Context 편집 패널, 정제 Context 미리보기, Context 적용, 선택형 브랜치 생성, 이전 메시지 수정” 정도로 제한하는 것이 타당한지 여쭙고 싶습니다.

10. 결론

FlowKit의 최종 방향은 LLM 자체를 개선하는 것이 아니라, **LLM에게 전달되는 대화 맥락을 사용자가 직접 관리할 수 있는 인터페이스를 구현하는 것이다**. 이 관점에서 가장 중요한 선행연구는 Mixed-Initiative Context와 ContextBranch이다. 전자는 Context를 사용자가 조작 가능한 상호작용 객체로 정의하고, 후자는 선형 대화의 context pollution을 줄이기 위한 브랜치 구조를 제안한다.

Lost in the Middle, LLMs Get Lost In Multi-Turn Conversation, MemGPT, Context Engineering survey는 FlowKit의 문제 정의와 시스템적 배경을 보강한다. ConstraintBench는 초기 제약 준수 아이디어의 배경으로는 의미가 있으나, 현재 FlowKit의 직접 Key Paper로 삼기에는 관련성이 낮다.

따라서 교수님 제출용 리서치에서는 다음과 같이 정리하는 것이 가장 적절하다.

FlowKit은 긴 대화에서 누적되는 불필요한 Context와 사용자 제어권 부족 문제를 해결하기 위해, 대화를 블록 단위로 선택·편집·정제하고 선택 Context 기반 브랜치를 생성하는 Context Editing Interface이다. 본 프로젝트는 Mixed-Initiative Context의 HCI 관점과 ContextBranch의 대화 버전관리 관점을 결

합하여, 학부 캡스톤 수준에서 구현 가능한 사용자 주도형 Context 관리 챗봇 프로토타입을 개발하는 것을 목표로 한다.

참고문헌

- [1] Haichang Li, Qinshi Zhang, Piaohong Wang, and Zhicong Lu. *Mixed-Initiative Context: Structuring and Managing Context for Human-AI Collaboration*. arXiv:2604.07121, 2026.
- [2] Bhargav Chickmagalur Nanjundappa and Spandan Maareshwari. *Context Branching for LLM Conversations: A Version Control Approach to Exploratory Programming*. arXiv:2512.13914, 2025.
- [3] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. *Lost in the Middle: How Language Models Use Long Contexts*. Transactions of the Association for Computational Linguistics, 12:157-173, 2024. DOI: 10.1162/tacl_a_00638.
- [4] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. *LLMs Get Lost In Multi-Turn Conversation*. arXiv:2505.06120, 2025.
- [5] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560, 2023; revised 2024.
- [6] Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. *A Survey of Context Engineering for Large Language Models*. arXiv:2507.13334, 2025.
- [7] Joseph Tso, Preston Schmittou, Quan Huynh, and Jibran Hutchins. *ConstraintBench: Benchmarking LLM Constraint Reasoning on Direct Optimization*. arXiv:2602.22465, 2026.